

Smart Rooftop Water Tank on a MongoDB Replica Set

CMP6207 Modern Data Stores — Final Report

Shekhar Lamichhane

30 September 2026

Abstract

A householder switches a rooftop pump on and walks away. The tank may overflow, the inlet may run dry, the pump may spin without moving water, or water may move while the pump is off. This report describes a document-store system that records such a household on a three-node MongoDB replica set (`rs0`, ports 27017–27019) and acts on it through a Node.js rule engine. The marked installation is home `home_001` with a 1000-litre roof tank `tank_roof_01`. It warns at 90% level while pumping, cuts the pump automatically at 98%, distinguishes a dry inlet from a failed pump, and escalates to `confirmed_fault` when an off command is ignored for 30 seconds. Sensor history is written with concern `w:1`; incidents use `w:majority`; all application reads prefer the primary. MQTT carries one message per device and QoS 1 commands; an embedded Aedes broker is used when port 1883 is free, otherwise an existing Mosquitto broker. An Express REST API (`tanks`, `devices`, `rules`, `telemetry`, `incidents`, `stats`, acknowledgement, `motors`, `notifications`, `health`) serves a polling status page and Telegram notifications. The same replica set also hosts a study-desk automation and a living-room climate automation in separate databases, reusing the ingest path and incident mechanism. All sensors are simulated (`normal`, `overflow`, `dry-run`, `pump-failure`, `abnormal-flow`, `fault`, `study`, `cool`); real hardware and a mobile application are out of scope.

Contents

1	Introduction	3
2	Types of NoSQL Databases and Selection of MongoDB	3
3	Relational and NoSQL Database Comparison	5
4	System Architecture	6
5	NoSQL Database Design	8
5.1	Dataset and Device Fleet	8
5.2	Document Structure	8
6	Implementation	9
6.1	End-to-End Runtime Flow	9
6.2	Software Installation and Commands	9
6.3	Implementation Evidence Screenshots	10

7	Data Management	10
8	REST API, Swagger and Dashboard	11
8.1	API Design	11
8.2	Dashboard Design	11
9	Testing	11
10	Distribution and Clustering	12
11	Security, Operations and Future Work	13
12	Conclusion	13

1 Introduction

The problem is deliberately ordinary. A rooftop tank holds 1000 litres. A person presses the pump on, leaves the roof, and returns later. Between those two moments nobody watches the level. If the return is late, water spills over the rim. If the municipal inlet is empty, the pump runs dry and wears itself out. If the pump is energised yet the level never rises, the pump has failed. If flow is measured while the pump is off, a valve leaks or a sensor has drifted. Each outcome needs a stored record, and two of them need a command sent back to the pump before the person climbs the stairs.

A relational table can hold these facts, but the household keeps changing shape. A level reading carries a percentage and a depth in centimetres; a flow reading carries litres per minute, a cumulative counter and a direction; a pump reading carries a state and a runtime; an inlet reading carries a single boolean. A desk lamp added later carries colour temperature and brightness; an air conditioner carries an operating mode. Forcing every variant into one fixed row means either a wide table full of nulls or a new table per device type, and both options make the most frequent query — “what is the current value of every device in this subsystem?” — into an assembly of joins across history.

The system built for this coursework therefore stores the household as documents. One **telemetry** collection per database keeps every reading with a polymorphic **reading** payload; one **device_latest** collection keeps exactly one current document per device, which is what the status page and the rule engine read. Rules themselves are documents: condition lists of **device.type**, **field**, **operator** and **value**, plus an action. Adding the desk and the living room required new databases and new rule documents, not a schema migration.

The marked scope is home **home_001**: roof tank **tank_roof_01** with four devices (**tank_level**, **water_flow**, **motor_state**, **source_presence**), study desk **focus_desk_01**, and living-room climate **climate_living_01**. The tank warns at 90% while the pump is on (incident only; the person may press Off on the status page at about 95% or at any level), and it switches the pump off automatically at 98% rather than waiting for 100%, because 100% is already an overflow. Dry-run, pump-failure, abnormal-flow and low-water (15% while the pump is off) each open incidents; a 30-second ignored off command escalates the open incident to **confirmed_fault**. Litres saved on a successful cutoff are estimated as current flow multiplied by 20 minutes of assumed unattended pumping.

The remainder of this report follows the required structure: the NoSQL landscape and the choice of MongoDB; a relational comparison; architecture; database design; implementation; data management; the REST API and dashboard; testing; distribution and failover on **rs0**; security, operations and future work; and a conclusion with Harvard references. Diagrams live in **plan/img/**; run logs, replica-set states and incident dumps live in **evidence/**.

2 Types of NoSQL Databases and Selection of MongoDB

NoSQL is not one technology but four broad families, each optimised for a different access pattern. A key-value store (for example Redis) maps opaque keys to opaque values with very low latency, which suits session caches and counters but offers little help when the application must query by device, subsystem, time range and severity. A wide-column store (for example Cassan-

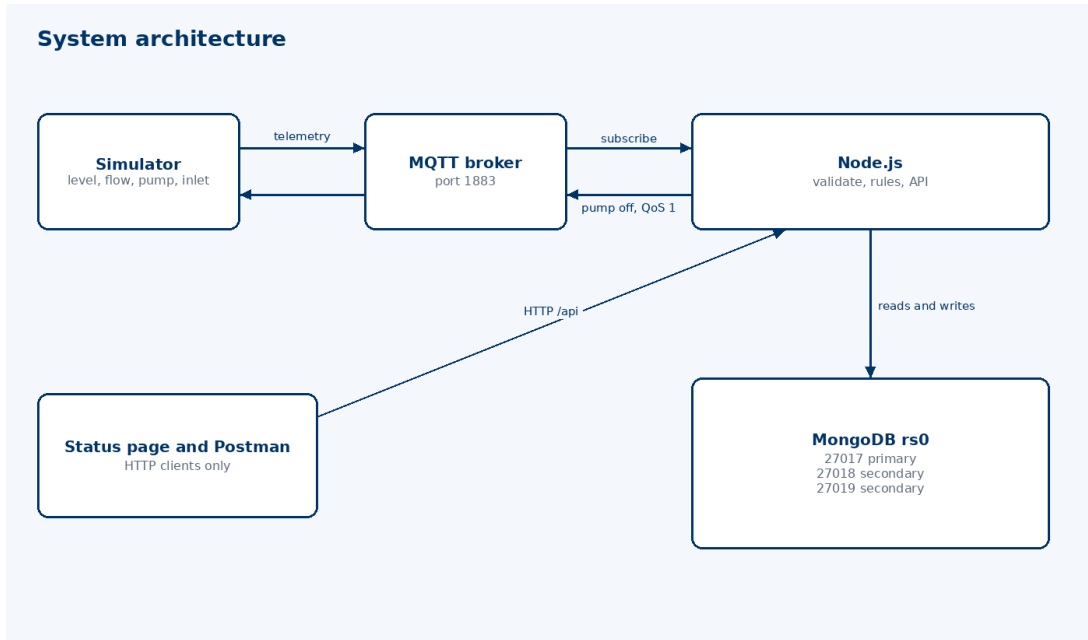


Figure 1: System architecture (placeholder): simulator → MQTT → Node.js ingest → MongoDB rs0 → rule engine → MQTT command; REST API → status page and Telegram. Source file: [plan/img/system-architecture.png](#).

dra or HBase) distributes massive write workloads across many nodes with tunable consistency, which suits telecom-scale event ingestion but imposes a rigid column-family discipline that fits uniform counters better than variant sensor payloads. A graph store (for example Neo4j) traverses relationships between nodes, which suits fraud rings or social recommendations but adds machinery this household never uses: devices relate to exactly one subsystem, and no multi-hop traversal drives any rule. A document store (for example MongoDB or Couchbase) keeps each record as a self-describing JSON-like document, indexes any field, and varies the shape of one document without touching its neighbours.

Time-series extensions cut across these families. InfluxDB and TimescaleDB compress timestamped numeric streams efficiently, and MongoDB itself offers time-series collections. They were considered and set aside for two reasons. First, the dominant read in this system is not a range scan but a point lookup of current state: the rule engine and the status page need the latest document per device on every cycle, which `device_latest` answers in one indexed query rather than a “latest per group” aggregation over history. Second, the interesting records — rules, incidents, notifications — are not numeric series at all but small operational documents with conditions, actions, escalation flags and delivery receipts. A pure series engine would force those back into a second store.

MongoDB was selected because the workload is document-shaped, query-shaped and availability-shaped in ways it answers directly. The polymorphic `reading` sub-document holds level, flow, pump, presence, lamp and climate payloads in one `telemetry` collection while per-type validation in the API keeps flexibility from becoming anarchy. Secondary indexes on `(subsystem_id, device_id, timestamp)` and on `device_latest(subsystem_id)` make both history charts and current-state joins cheap. Condition-list rules map naturally onto embedded arrays, and one open incident per rule is enforced with a partial unique index rather than application locking.

The replica set supplies the availability mechanism the module assesses: three voting members, automatic election, majority write concern for the records that must survive a primary loss, and a lighter `w:1` for replaceable sensor history. Mongoose supplies schema validation, discriminated models per database, and `useDb` so one Node process serves three databases on the same set. Finally, the surrounding ecosystem — the Node.js driver, Express, Mosquitto/Aedes MQTT clients, Compass and `mongosh` — made every piece of required evidence (ingest logs, incident dumps, `rs.status()` transitions) collectable without extra infrastructure.

The alternatives were rejected on fit, not on quality. A key–value store cannot answer “all open overflow incidents for this home, newest first” without secondary indexing built by hand. A wide-column store would handle the write rate easily yet complicate the small-document operational workload and the ad-hoc coursework queries. A graph store would model the home–subsystem–device tree elegantly but never traverse it deeply enough to justify the engine. A dedicated series database would compress history well but split the system into two stores for no availability gain. MongoDB keeps one store, one query language and one replication story for the whole household.

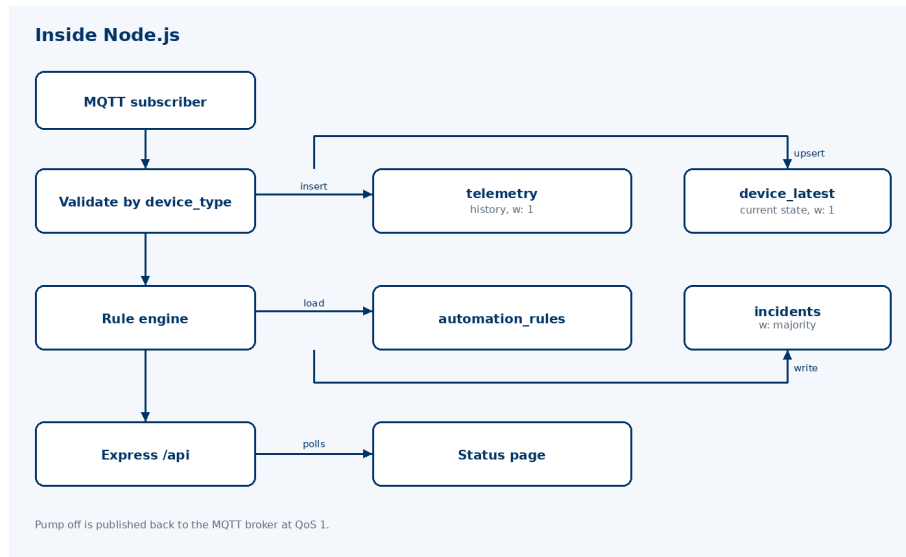


Figure 2: Inside Node.js (placeholder): validation → `telemetry` insert (`w:1`) → `device_latest` replace → subsystem rule evaluation → incident (`w:majority`) + MQTT command. Source: `plan/img/node-flow.png`.

3 Relational and NoSQL Database Comparison

A relational design for this household is entirely possible, and its strengths should be stated fairly. Tanks, homes, devices, rules, incidents and notifications are well-bounded entities with stable keys; foreign keys would guarantee that no reading references a nonexistent device, and a single SQL statement could join tanks to devices to latest readings for a report. Multi-row ACID transactions would protect operations the business could not afford to half-complete, such as billing or stock decrements. The query language is standard, the tooling is mature, and any competent developer can read the schema on day one.

The friction appears exactly where the coursework lives: variant readings, evolving rules, and

the “current state” read pattern. Three relational options exist for the reading, and each hurts. Option one is a single wide `reading` table with columns for every device type; most rows are then mostly null, every new sensor adds a column, and check constraints multiply. Option two is an entity–attribute–value table with one row per measured attribute; the schema never changes, but reconstructing one flow reading means pivoting several rows, and type safety moves into application code. Option three is a table per device type (`level_reading`, `flow_reading`, `pump_reading`, ...); the schema is clean, but the status page must union or join N tables on every two-second poll, and adding the desk lamp means a DDL migration plus a new join branch. The document design avoids all three: one `telemetry` collection, one polymorphic `reading` field, validation per `device_type` in `validateReading.js`, and indexes that serve both history and latest-state queries.

Rules widen the gap further. A water rule tests level, pump state, inlet presence and flow together; a desk rule tests presence, ambient light and lamp state; a climate rule tests temperature against the air-conditioner mode with a hysteresis band (cool above 28°C while off, stop at or below 26°C while cooling). In a relational schema each predicate shape wants its own columns or its own rule-type table, so every new automation revisits the DDL. Here each rule in `automation_rules` is a condition list of `{device_type, field, operator, value}` plus an `action` document; the desk and climate automations were added as data, seeding `rule_study_lamp`, `rule_lamp_off`, `rule_cool_on` and `rule_cool_off` without touching existing collections.

Consistency choices also differ by design rather than by default. A relational system typically applies the same transactional guarantee to every write. This system deliberately splits durability: high-frequency telemetry and `device_latest` use `w:1` because a single lost sensor sample is replaced by the next one seconds later, while incidents use `w:majority` because an overflow cutoff or a confirmed fault must still be present after a primary failure. Reads use `primary` so the engine never decides from a lagging secondary. The failover run in `evidence/failover-election.log` and `evidence/rs-status-failover.json` shows why: after the 27018 primary was shut down, 27017 was elected within about three seconds (electionTime delta) and the cutoff incident written before the failure remained retrievable (see `evidence/failover-incident.json`).

Where the relational model still wins is integrity across entities: nothing in MongoDB stops a telemetry insert naming an unknown device except the application validator, whereas a foreign key would stop it at the store. That is accepted consciously. The validator, the per-topic subsystem check (message body must match the topic path), and the single-writer ingest path provide the practical guarantee, and the payoff is a store that absorbs new device types as documents rather than migrations. For an IoT household whose sensors change faster than its accounts, that trade favours documents.

4 System Architecture

The runtime has five stages. Simulators stand in for sensors and publish one MQTT message per device on topics of the form `ioThings/{home_id}/{subsystem_id}/{device_type}/{device_id}/telemetry`. The broker is the embedded Aedes instance on port 1883 when that port is free; if a system Mosquitto already holds it, the API attaches to that broker instead, so simulator commands never change. The Node.js ingest path (`ingest.js`, `mqtt.js`) parses the topic, rejects bodies

whose subsystem disagrees with the path, validates the payload by `device_type`, inserts into `telemetry` at `w:1`, replaces the matching `device_latest` document, and then runs the rule engine for that subsystem only (water evaluator versus room evaluator). Commands return along `.../command` at QoS 1: `motor_state` off/on for the pump, `study/off` for the lamp, `cool/off` for the climate unit. The status page (`backend/src/public/index.html` served by Express at `http://localhost:3000`) polls the API every two seconds and never touches MongoDB or MQTT directly; manual Off/On buttons travel through `POST /motors/:deviceId/override`, which records a `manual_override` incident, stores a notification, and publishes the command. Figure 1 summarises the loop; Figure 3 traces the overflow path.

Persistence sits behind the API on the replica-set URI with `readPreference=primary`. One Mongoose connection fans out through `useDb` to the three databases, so the subsystem type found by `resolveSubsystem` selects the correct models for every read and write. Because history (`telemetry`) and current state (`device_latest`) share the same key shape, the join the page needs on every poll is a small indexed query by `subsystem_id` rather than an aggregation over time series. Notifications close the loop to the person: each incident writes exactly one notification document, and the Telegram sender delivers the same text only when both the bot token and the chat identifier are configured, otherwise marking the receipt as on-page delivery.

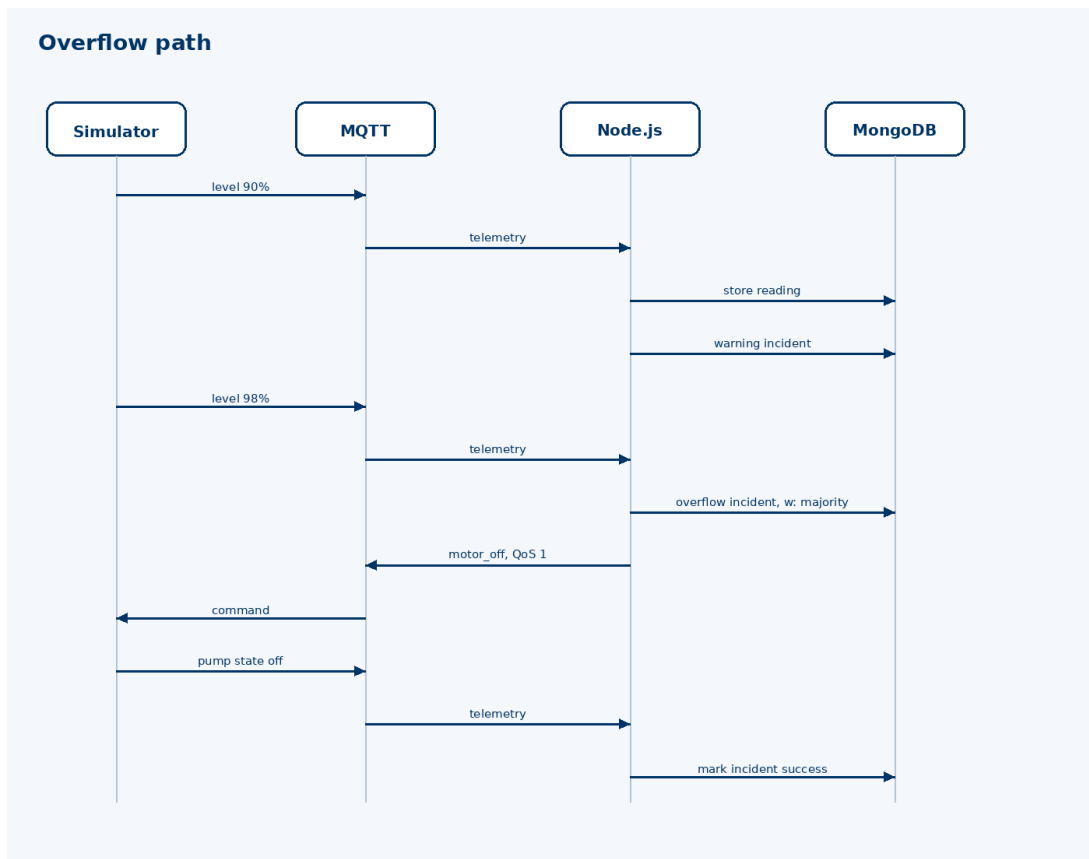


Figure 3: Overflow path (placeholder): 90% warning → 98% cutoff command → incident (w:majority) + Telegram → simulator applies off. Source: `plan/img/overflow-sequence.png`.

5 NoSQL Database Design

5.1 Dataset and Device Fleet

Three databases share replica set `rs0`: `smart_water`, `smart_focus` and `smart_climate`, selected per subsystem through `useDb`. Only `smart_water` holds a `tanks` collection (seeded `tank_roof_01`, capacity 1000 L, thresholds 90/98/15). Every database holds `subsystems`, `devices`, `telemetry`, `device_latest`, `automation_rules`, `incidents` and `notifications`; every document carries `subsystem_id` so a water reading can never land in the climate collections and home-wide lists merge the three stores sorted by time. The water fleet is four devices: `tank_level_roof_01` (`level_pct`, `level_cm`), `water_flow_roof_01` (`flow_rate_lpm`, `cumulative_litres`, `direction`), `motor_roof_01` (`state`, `runtime_seconds`), and `source_roof_01` (`water_present`). The desk adds presence, ambient-light and lamp devices; climate adds temperature and air-conditioner devices.

5.2 Document Structure

A telemetry document `{home_id, subsystem_id, tank_id?, device_id, device_type, reading, timestamp}` keeps routable fields at top level for indexing and nests only the variant payload inside `reading`. `device_latest` mirrors the same shape with one document per device, updated by replacement. A rule document holds `rule_id`, `subsystem_id`, an `enabled` flag, a `conditions` array and an `action`; multi-sensor water rules additionally require the referenced readings to be mutually current, so a stale flow value from before a pump transition cannot fire them, and at most one open incident per `rule_id` is kept (an old open fault otherwise masks a new overflow until acknowledged). An incident records `severity`, `triggered_by` snapshots, `action_taken` with status (`success/pending/cleared`), and, for cutoffs, `litres_saved` = `flow` × 20 with the assumed 20 unattended minutes stored alongside so the estimate is auditable. Derived figures — litres in tank (`capacity` × `level`/100), litres and minutes to the 98% cutoff (only while pumping with positive flow) — are computed on read in `tankService.js` and never stored, so they cannot drift from their source sensors. Key indexes cover telemetry time queries, latest-state lookups, and open-incident guards.

Severity	Fires when	Action
warning	level $\geq$ 90% and pump on	incident only (person may press Off $\approx$ 95%)
overflow_prevented	level $\geq$ 98% and pump on	pump off + incident; litres_saved = flow × 20
dry_run	pump on and inlet absent	pump off + incident
pump_failure	pump on, water present, $\approx$ 0 flow, level flat	incident only
abnormal_flow	pump off but flow > 0	incident only
low_water	level $\leq$ 15% and pump off	incident only
confirmed_fault	off sent, pump still on after 30 s	escalate open incident
manual_override	Off/On pressed on page	MQTT command + incident

The inlet boolean is the diagnostic pivot: dry-run and pump-failure both show a pump that is on with a non-rising level, but only dry-run shows `water_present` false.

6 Implementation

6.1 End-to-End Runtime Flow

Each cycle is identical across subsystems. The simulator publishes a per-device reading; ingest validates, writes history, refreshes `device_latest`, and evaluates only the enabled rules of that subsystem against current values. The water evaluator checks the warning, cutoff, dry-run, pump-failure, abnormal-flow and low-water rules in severity order; the room evaluator checks the lamp pair (present $\rightarrow$ `lamp_study` at 6500 K/100%; absent $\rightarrow$ `lamp_off`, skipped when already in target state) and the climate pair with its 26/28°C hysteresis so the unit does not chatter at one threshold. Firing opens an incident at `w:majority`, stores a notification, optionally publishes a QoS 1 command, and — when Telegram credentials are configured — sends the same text to the configured chat. The simulator then applies the command and publishes the resulting device state, which is what the next page poll displays. The 30-second `confirmed_fault` timer watches every off command: if the pump still reports on, the open cutoff incident is marked escalated rather than opening a duplicate.

Concretely the water cycle is: level, flow, pump and inlet messages arrive within the same second; `ingest.js` writes four telemetry documents and replaces four latest-state documents; `rules/engine.js` loads the enabled rules for `tank_roof_01` from `automation_rules` and tests their condition lists in `rules/conditions.js`; a firing rule calls `rules/actions.js`, which publishes the MQTT command and persists the incident plus notification through `services/`; the simulator receives the command, updates its internal pump flag, and its next telemetry batch reflects the new state, closing the feedback loop. Desk and climate cycles reuse the same files with different evaluators and command vocabularies, which is why one developer can reason about all three automations from a single code path.

6.2 Software Installation and Commands

Prerequisites are Node.js 18+, a local `mongod` triple on 27017–27019 initialised once as `rs0` (never re-initiate), and optionally Mosquitto. Setup is `npm install`, `scripts/replica-up.sh`, `npm run seed` (three subsystems, devices, rules), `npm run dev` (API + Aedes on 1883 if free). Simulation is `npm run simulate -- <normal|overflow|dry-run|pump-failure|abnormal-flow|fault|study|cool>`; only one tank scenario runs at a time while `study` and `cool` may overlap the tank on disjoint topics. `cool` loops between heating (+0.6°C/s while off) and cooling (−0.5°C/s to a 24°C floor) and must be stopped after the recorded cycle. Telegram needs `TELEGRAM_BOT_TOKEN` and `TELEGRAM_CHAT_ID` in `.env` before the API starts; without them notifications are still stored and shown on the page as on-page only.

The seed writes fixed identifiers so evidence is reproducible: home `home_001`, tank `tank_roof_01` at 1000 litres, pump `motor_roof_01`, and the full rule set (`rule_warning_roof`, `rule_overflow_roof`, `rule_dry_run_roof`, `rule_pump_failure_roof`, `rule_abnormal_flow_roof`, `rule_low_water_roof`, plus the desk and climate rules). The `normal` scenario fills while pumping and drains after switch-off, exercising the falling-level path that clears the low-water incident; `overflow` climbs 80% to 98% and holds; `dry-run` publishes inlet absent with the pump on; `pump-failure` publishes inlet present with near-zero flow and a flat level; `abnormal-flow` publishes flow while the pump is off; `fault` acknowledges no command so the escalation timer can be observed.

6.3 Implementation Evidence Screenshots

To be captured for the appendix: API process log (`evidence/api-process.log`, health in `evidence/api-health.json`); overflow simulator trace (`evidence/overflow-simulator.log`) showing the 80% $\rightarrow$ 98% climb and the cutoff incident with `litres_saved` 300 ($15 \text{ L/min} \times 20$); fault trace (`evidence/fault-simulator.log`, `evidence/confirmed-fault-incident.json`) showing the ignored off command and escalation; notification dumps (`evidence/fault-notifications.json`); replica states before, during and after failover (`evidence/rs-status-before.json`, `evidence/rs-status-failover.json`, `evidence/rs-status-rejoined.json`, `evidence/failover-election.log`).

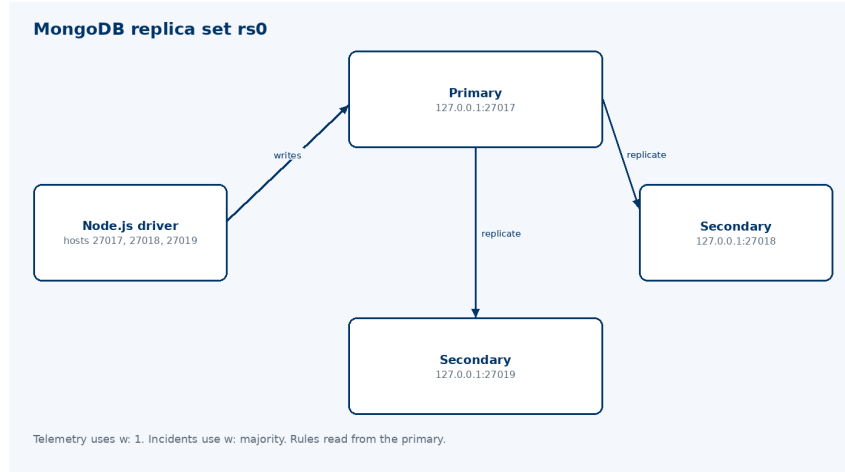


Figure 4: Replica set `rs0` (placeholder): primary 27018 $\rightarrow$ failover to 27017 $\rightarrow$ rejoin as secondary. Source: `plan/img/replica-set.png`.

7 Data Management

Data quality is enforced at the boundary, not wished for afterwards. Every ingest passes three gates: the topic parses to the four-part path, the body names the same `subsystem_id`, and `validateReading.js` accepts the payload shape for its `device_type` (ranges for level and flow, enum states for pump, lamp and climate). Failures are rejected before any write, so malformed simulator output cannot poison history or trigger rules. Freshness is enforced inside the engine: multi-sensor rules compare the timestamps of the readings they consume and refuse to fire on a mixture of new pump state with old flow. Lifecycle policy keeps history append-only while `device_latest` is a bounded working set of one document per device, so read cost is independent of retention. Derived values (litres, minutes to cutoff) are recomputed on read. Notifications are write-once records with a delivery receipt (`channel`, `delivered`); Telegram is a best-effort sender over a durable store, meaning a missing token or rejected chat never loses the alert — it only changes the receipt to on-page. Finally, incident closure is explicit: overflow incidents resolve when the command succeeds, low-water clears when the level recovers or the pump starts, and faults stay open until acknowledged through the API, which is why at most one open incident per rule is the correct guard.

8 REST API, Swagger and Dashboard

8.1 API Design

The Express API at `http://localhost:3000/api` exposes `GET /health` (process, broker, replica members), full CRUD for `/tanks`, `/devices` and `/rules`, `POST /telemetry` plus range queries, `GET /incidents` with `/incidents/stats` and per-incident `PATCH` acknowledgement, `POST /motors/:deviceId` for manual pump commands, notification listing, and replica-status reporting. Semantics are uniform: collection GETs paginate and filter by subsystem, POSTs validate before writing, and incident writes carry `w:majority` while telemetry writes carry `w:1`. Manual overrides are first-class incidents rather than silent publishes, so the audit trail shows who (page) commanded what and when.

8.2 Dashboard Design

The single-file status page opens as a room launcher (Water, Light, Climate, Alerts) and polls every two seconds. The water view shows level, litres, flow, pump and inlet state, minutes-to-cutoff while pumping, history chart, running totals (incidents, litres saved), and acknowledge buttons; the desk and climate views show presence/lamp and temperature/AC state respectively. Because the page reads only the API, every number on screen is proof that MQTT → MongoDB → REST is working rather than a hard-coded mock.

API and dashboard evidence to attach: health JSON (`evidence/api-health.json`), incident summary (`evidence/incidents-summary.log`), notification summary (`evidence/fault-notifications-summary.log`) and page screenshots for the warning banner ($\approx 92\%$), the cutoff state (98%, pump off, 300 L saved), the confirmed-fault escalation, and the desk/climate views. Swagger-style request/response captures for `/health`, `/devices`, `/telemetry`, `/incidents` and `/incidents/stats` belong alongside them; where formal Swagger UI is unavailable, `mongosh` dumps and browser captures serve the same evidential role.

9 Testing

Testing is scenario-based against the live stack. Each simulator case is a test: **normal** must fill then drain with no cutoff incident; **overflow** must raise a 90% warning then a 98% cutoff with pump off and 300 L saved; **dry-run** and **pump-failure** must open their respective incidents with the inlet boolean as the only distinguishing snapshot; **abnormal-flow** must fire while the pump is off; **fault** must escalate to **confirmed_fault** after 30 s; **study** must drive the lamp to 6500 K/100% then off; **cool** must start cooling above 28°C and stop at 26°C. Pass criteria are checked in `mongosh` (incident documents, `triggered_by` snapshots, action statuses) and on the page (banners, totals, acknowledgement). Recorded runs already show the overflow warning/cutoff pair, the dry-run/pump-failure/abnormal-flow incidents, and the fault escalation with two Telegram-marked notifications.

Case	Must open	Must show
normal	low-water only at $\leq 15\%$	level falls after pump stops
overflow	warning + cutoff	pump off, 300 L saved
dry-run / pump-failure	respective incident	inlet snapshot differs
abnormal-flow	abnormal-flow	no pump command
fault	escalation after 30 s	open incident marked escalated
study / cool	lamp / climate incidents	actuator follows state

Full automation under **npm test** and negative tests (malformed payloads, stale-timestamp mixes, double-acknowledgement) remain future work; the evidence logs above are the current proof set. Regression discipline is procedural for now: re-seed, run each tank scenario alone, confirm the incident set in **mongosh** matches the table, and only then record screenshots, because a stray second simulator on the same topics would interleave readings and invalidate the run.

10 Distribution and Clustering

Distribution in this project means a replica set, and the coursework asks that its behaviour be demonstrated rather than asserted. A replica set keeps identical data on several **mongod** processes, elects one primary to take writes, replicates the operation log to secondaries, and elects a new primary automatically when the old one disappears, provided a majority of voting members remains. Three members on 27017–27019 is the smallest configuration that tolerates one failure while still reaching a majority of two; a single-node “replica set” would prove configuration syntax but nothing about availability, so three local processes are used.

Configuration is straightforward. Each member runs with **--replSet rs0** and a distinct port and data directory; the set is initiated once naming all three members; the application connects with **mongodb://127.0.0.1:27017,127.0.0.1:27018,127.0.0.1:27019/smart_water?replicaSet=rs0** and **readPreference=primary**. All application reads specify the primary so rules never act on replicated-lag state. Telemetry and **device_latest** write at **w:1**: the write returns once the primary has it, which keeps per-second sensor traffic fast at the cost that a primary crash in the same instant could lose that one sample — acceptable because the next sample arrives seconds later. Incidents write at **w:majority**: the write returns only once two of three members hold it, so a cutoff or fault record acknowledged to the engine survives the loss of any single node. That split is the central availability argument of the report: cheap speed where data is replaceable, majority durability where data is the safety record.

The health endpoint exposes the set truthfully. When connected to **rs0** it returns the set name, each member’s address, state string and health flag; when pointed at a standalone server it says so instead of inventing members. **scripts/replica-status.sh** and the **rs.status()** captures provide the same information out of band.

Failover was exercised, not assumed. With the system mid-overflow, the primary on 27018 was shut down at 17:21:39Z; the next poll showed 27017 as primary and 27018 unreachable with 27019 secondary, and electionTime comparison places the election about three seconds after the shutdown command (the longer wall-clock span in the log includes the shutdown handshake). The cutoff incident written before the failure (**inc_1790788742795**, **flow_rate_lpm** 15, **litres_saved** 300) remained readable through the new primary, and the fault notification

pair for the same window was intact; the stopped member was restarted and rejoined as a secondary, confirmed in `rs-status-rejoined.json` and `replica-status-final.log`. During the window without a primary, sensor writes pause rather than silently diverging, which is correct primary-preferring behaviour for a safety system: no split-brain cutoff, no lost majority incident.

Backup, recovery and scale complete the picture. `mongodump` against the primary (or a secondary with `--oplog` for point-in-time capture) backs all three databases; restore replays into a fresh set, and the seed script rebuilds subsystem/device/rule fixtures deterministically if a logical rebuild is preferred. Sharding is not deployed — three replicas of a household workload do not need it — but the shard-key choice is recorded for the high-volume collection: `telemetry` would shard on `{subsystem_id: 1, timestamp: 1}` (or `{device_id: 1, timestamp: 1}`), keeping each subsystem’s history together while distributing time ranges, and `device_latest` would stay unsharded as a small lookup collection. Production would separate the members onto different hosts or availability zones; co-location here is a coursework constraint, and the election logic demonstrated is identical either way.

Listing 1: Replica URI used by server and evidence (placeholder for screenshot).

```
MONGO_URI=mongodb://127.0.0.1:27017,127.0.0.1:27018,127.0.0.1:27019/
smart_water?replicaSet=rs0
```

11 Security, Operations and Future Work

This is a prototype, and its trust boundaries are stated openly. MongoDB runs without authentication, the broker accepts anonymous local clients, the API exposes no login, and Telegram credentials sit in `.env`. That is acceptable on an isolated coursework machine and unacceptable anywhere else. Production would enable MongoDB keyfile/SCRAM authentication with role-separated users, MQTT username/password plus TLS, API authentication and rate limiting, secrets management instead of flat files, and validation already present extended with JSON-schema collection validators. Operationally the system is already well-behaved: replica scripts for up/down/status, a health endpoint that reports broker and set state, append-only history with a bounded latest-state set, and explicit acknowledgement so stale open faults cannot mask new overflows silently. Future work is a phone client on the existing API, automated tests with negative cases, formal Swagger UI, per-home multi-tenancy, and sharded telemetry if device counts grow by orders of magnitude.

MongoDB and distribution evidence to attach: Compass views of `telemetry`, `device_latest`, `automation_rules`, `incidents` and `notifications`; index listings; the three `rs.status()` captures; and the election/rejoin logs. Items still requiring fresh screenshots are marked [CAPTURE] in the appendix checklist below.

12 Conclusion

One household, three document databases, one replica set. MQTT brings each reading in and carries each command out; the rule engine turns current state into incidents with majority durability; the page and Telegram turn incidents into human action. The roof tank warns at

90%, stops itself at 98% with an auditable litres-saved estimate, tells dry-run from pump-failure by the inlet sensor, and escalates ignored commands after 30 seconds. Desk and climate reuse the same path with their own rules. The evaluation shows the document model fits variant readings and condition-list rules better than fixed tables, the write-concern split protects safety records without slowing sensor traffic, and the failover run proves the cutoff survives a primary loss. No new sensor requires a migration, no safety record depends on a single node, and no alert depends on a single delivery channel. The remaining work is evidence capture, above all the dashboard, Compass and Swagger screenshots, rather than any change to that loop.

References

- [1] Banks, A. and Gupta, R. (eds.) (2014) *MQTT Version 3.1.1*. OASIS Standard. Available at: <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html> [Accessed 30 September 2026].
- [2] MongoDB, Inc. (2026a) *Replication and replica sets*. MongoDB Manual. Available at: <https://www.mongodb.com/docs/manual/replication/> [Accessed 30 September 2026].
- [3] MongoDB, Inc. (2026b) *Write concern*. MongoDB Manual. Available at: <https://www.mongodb.com/docs/manual/reference/write-concern/> [Accessed 30 September 2026].
- [4] MongoDB, Inc. (2026c) *Indexes*. MongoDB Manual. Available at: <https://www.mongodb.com/docs/manual/indexes/> [Accessed 30 September 2026].
- [5] MongoDB, Inc. (2026d) *Data modeling: flexible schema*. MongoDB Manual. Available at: <https://www.mongodb.com/docs/manual/core/data-modeling-introduction/> [Accessed 30 September 2026].
- [6] OpenJS Foundation (2026) *Express — Node.js web application framework*. Available at: <https://expressjs.com/> [Accessed 30 September 2026].
- [7] Eclipse Foundation (2026) *Eclipse Mosquitto documentation*. Available at: <https://mosquitto.org/documentation/> [Accessed 30 September 2026].
- [8] Strauch, C. (2011) *NoSQL databases*. Stuttgart: Stuttgart Media University. Available at: <https://www.christof-strauch.de/nosql dbs.pdf> [Accessed 30 September 2026].
- [9] Sadalage, P. and Fowler, M. (2012) *NoSQL distilled: a brief guide to the emerging world of polyglot persistence*. Boston: Addison-Wesley.

Appendix A — Evidence checklist

Figure	File	State
Architecture diagram	plan/img/system-architecture.png	present
Node ingest flow	plan/img/node-flow.png	present
Replica-set diagram	plan/img/replica-set.png	present
Overflow sequence	plan/img/overflow-sequence.png	present
API health	evidence/api-health.json	present
Overflow trace	evidence/overflow-simulator-logs.log	present
Fault trace + escalation	evidence/fault-simulator-logs.log	present
	evidence/confirmed-fault-incident.json	present
Notifications — evidence/fault-notifications.json		present
Replica states + election	evidence/rs-status-before-prison.json, evidence/rs-status-failover.json, evidence/rs-status-rejoined.json, evidence/failover-election.log	present
Dashboard screenshots (warning/cutoff/fault/desk/climate)	evidence/	[CAPTURE]
Compass collection + index screenshots	evidence/	[CAPTURE]
Swagger/response captures	evidence/	[CAPTURE]